Markov 3_item state prediction

Importing relevant libraries

```
import pandas as pd
import numpy as np
from collections import defaultdict
```

Reading the file and only taking few columns for analysis

```
filename = 'full_file.csv'  # Replace with your CSV path
df = pd.read_csv(filename)

# Required columns
required_cols = ['user_id', 'order_id', 'product_id', 'add_to_cart_order',
'order_dow', 'order_hour_of_day']
for col in required_cols:
    if col not in df.columns:
        raise ValueError(f"Missing column: {col}")
```

Creating lag features within each order

Lag features are previous 3 product in an order

```
# 2 Sort and Create Lag Features (shift only within orders)
# ----------------------------
df = df.sort_values(['user_id', 'order_id', 'add_to_cart_order'])
df['lag1'] = df.groupby(['user_id', 'order_id'])['product_id'].shift(1)
df['lag2'] = df.groupby(['user_id', 'order_id'])['product_id'].shift(2)
df['next_product'] = df.groupby(['user_id', 'order_id'])['product_id'].shift(-1)

df = df.dropna(subset=['lag1', 'lag2', 'next_product'])
df[['lag1','lag2','next_product']] = df[['lag1','lag2','next_product']].astype
(int)
```

Splitting data in test and train, we are building a data frame with unique order_id so as to avoid leakage of information between train & test

80% of the orders are used for training and 20% for test. Output shown below indicate size of test and train

```
# 3 Train/Test Split by order (no leakage)
# ----------------------------
orders = df['order_id'].unique()
np.random.seed(42)
np.random.shuffle(orders)
train_orders, test_orders = np.split(orders, [int(len(orders) * 0.8)])

train_df = df[df['order_id'].isin(train_orders)].copy()
test_df = df[df['order_id'].isin(test_orders)].copy()

print(f"Train samples: {len(train_df)}, Test samples: {len(test_df)}")
```

```
Train samples: 6423539, Test samples: 1607167
```

Building transition probabilities, what is being evaluated by function is

$$P(next_product|previous_products, dow, hour)$$

3-Item Context Model

$$P(next|lag2, lag1, current_product, dow, hour)$$

2-Item Context Model

$$P(next|lag1, current_product, dow, hour)$$

1-Item Model (no context)

$$P(next|current_product)$$

Global Popularity Fallback

Global Popularity Fallback is a backup recommendation strategy used when your model doesn't have enough information to predict the next item from the sequence.

How is it calculated

Free offer ×

Log

1 Step 1 — Count how often each product appears

From your dataset you take the column:

```
next_product
```

Example data:

order	product_id	next_product
1	milk	bread
2	bread	butter
3	milk	bread
4	eggs	milk
5	bread	butter

Now count occurrences of **next_product**.

Result:

next_product	count
bread	2
butter	2
milk	1

2️⃣ Step 2 — Convert counts into probabilities

We divide each count by **total transitions**.

Total transitions:

```
2 + 2 + 1 = 5
```

Probability formula:

$$P(product) = \frac{\text{count of product}}{\text{total next products}}$$

Result:

product	count	probability
bread	2	0.40
butter	2	0.40
milk	1	0.20

All the above is done through code written below

```python
# 4️⃣ Build Transition Counts and Probabilities (context-aware)
# ----------------------------
def build_transition_probs_with_context(df, group_cols):
    counts = df.groupby(list(group_cols) + ['order_dow','order_hour_of_day',
    'next_product']).size().reset_index(name='count')
    counts['prob'] = counts['count'] / counts.groupby(list(group_cols)
    +['order_dow','order_hour_of_day'])['count'].transform('sum')
    return counts

# 3-item + context
counts_3_ctx = build_transition_probs_with_context(train_df, ['lag2','lag1',
'product_id'])
# 2-item + context
counts_2_ctx = build_transition_probs_with_context(train_df, ['lag1',
'product_id'])
# 1-item (no context)
counts_1 = train_df.groupby(['product_id','next_product']).size().reset_index
(name='count')
counts_1['prob'] = counts_1['count'] / counts_1.groupby('product_id')['count'].
transform('sum')
# Global popularity fallback
global_counts = train_df['next_product'].value_counts(normalize=True).
reset_index()
global_counts.columns = ['product_id','prob']
```

This code converts your probability tables into fast lookup dictionaries so the recommender system can quickly predict the next product during inference.

In short:

It turns data frames of probabilities into Python dictionaries for fast access.

Think of it as building a search index.

```python
# 5 Build Lookup Dictionaries for Fast Prediction
# ----------------------------
def build_lookup_dict_with_context(df, key_cols):
    d = defaultdict(list)
    for row in df.itertuples(index=False):
        key = tuple(getattr(row, col) for col in key_cols) + (row.order_dow,
        row.order_hour_of_day)
        d[key].append((row.next_product,row.prob))
    for key in d:
        d[key].sort(key=lambda x: x[1], reverse=True)
    return d

dict_3_ctx = build_lookup_dict_with_context(counts_3_ctx, ['lag2','lag1',
'product_id'])
dict_2_ctx = build_lookup_dict_with_context(counts_2_ctx, ['lag1','product_id'])

dict_1 = defaultdict(list)
for row in counts_1.itertuples(index=False):
    dict_1[(row.product_id,)].append((row.next_product,row.prob))
for key in dict_1:
    dict_1[key].sort(key=lambda x: x[1], reverse=True)

global_list = global_counts.values.tolist()  # [(product_id,prob),...]
```

Looking up next product

This function predicts the next product a user will buy using the lookup dictionaries you built earlier. It follows a hierarchical backoff strategy: it tries the most specific model first, and if that fails it falls back to simpler models.

In simple terms:

Use the most detailed history possible to predict the next product.

```python
# 6 Prediction Function with Backoff & Context
# ----------------------------
def predict_next_product(p2, p1, p0, order_dow, order_hour, top_k=5):
    # 3-item + context
    key3 = (p2,p1,p0,order_dow,order_hour)
    if key3 in dict_3_ctx:
        return [p[0] for p in dict_3_ctx[key3][:top_k]]
    # 2-item + context
    key2 = (p1,p0,order_dow,order_hour)
    if key2 in dict_2_ctx:
        return [p[0] for p in dict_2_ctx[key2][:top_k]]
    # 1-item
    key1 = (p0,)
    if key1 in dict_1:
        return [p[0] for p in dict_1[key1][:top_k]]
    # Global fallback
    return [p[0] for p in global_list[:top_k]]
```

Testing the logic

```python
# 7️⃣ Evaluation (sample 30k rows for speed)
# ----------------------------
test_sample = test_df.sample(min(30000,len(test_df)), random_state=42)

top_k = 5
hits = 0
precision_sum = 0
reciprocal_rank_sum = 0
predicted_items = set()

for row in test_sample.itertuples(index=False):
    preds = predict_next_product(row.lag2,row.lag1,row.product_id,row.order_dow,
    row.order_hour_of_day,top_k)
    predicted_items.update(preds)
    if row.next_product in preds:
        hits += 1
        rank = preds.index(row.next_product)+1
        reciprocal_rank_sum += 1/rank
        precision_sum += 1/top_k
    else:
        precision_sum += 0

n = len(test_sample)
hit_rate = hits/n
precision = precision_sum/n
mrr = reciprocal_rank_sum/n
recall = hit_rate
coverage = len(predicted_items)/train_df['product_id'].nunique()
```

Based on the above we have obtained evaluation metrics

```
Evaluation Metrics (Top-5):
HitRate@5: 0.0661
Precision@5: 0.0132
MRR: 0.0389
Recall@5: 0.0661
Coverage@5: 0.2264
```

What each value means

HitRate@5 = 0.0661

- The correct item appears in the top 5 recommendations 6.6% of the time.
- For Instacart where users have 50k+ products, this is better than random, but still weak.

Random baseline approximation:

$$5/50{,}000 \approx 0.0001$$

So 0.066 >> random, but still not strong enough for a production recommender.

Precision@5 = 0.0132

Precision@5 = 1.32%

Meaning:

- Out of 5 recommendations,
- ~0.066 correct items per prediction

Equivalent interpretation:

~1 correct item every 75 recommendations

That’s low but typical for simple sequential models.

Recall@5 = 0.0661

Same as HitRate because there is 1 relevant item per test instance.

So:

- 6.6% of true next items are captured in top-5

Again, modest.

MRR = 0.0389

Mean Reciprocal Rank measures how high the correct item appears.

0.0389 roughly implies:

average rank ≈ 1 / 0.0389 ≈ 25

Meaning the correct item tends to appear around rank ~25.

For a top-5 evaluation, that’s not great.

Coverage@5 = 0.2264

Coverage = 22.6% of items appear in recommendations

This is actually reasonable.

It means your model isn't recommending only the same popular items, which is good.

Our model uses states like:

state = last product
state = last category
state = last basket pattern

Markov models assume:

P(next_item | current_state)

But **Instacart behaviour depends on:**

- user preferences
- basket composition
- reorder habits
- long-term history

So **Markov models struggle here**.

Typical performance comparisons on Instacart-like datasets:

Model	HitRate@5
Popularity baseline	~0.02–0.04
Markov chain	~0.05–0.08
ItemCF	~0.10–0.15
RNN / GRU4Rec	~0.15–0.25
Transformer	~0.20–0.35

Your 0.066 fits exactly in the Markov range.

So methodologically it's reasonable.